Enterprise Cloud & Headless E-Commerce

Virto Commerce & AWS Cloud Deployment

Technical Submission, Architecture Defense & Runbook

Candidate:	Technical Candidate Submission
Platform:	Virto Commerce 3.x (Headless Vue 3 B2B Architecture)
Infrastructure:	Docker, AWS Emulation (Floci), HashiCorp Terraform
Date:	November 2026
Status:	Complete, Tested & Verified (All 11 Proof Milestones)

1. Executive Overview & Technical Investigation

I wanted to follow up on my previous submission regarding the Virto Commerce Storefront deployment.

In my initial review, the legacy storefront container encountered a routing exception on legacy REST endpoints (`/api/stores`). Rather than stopping at that legacy hurdle, I conducted a deeper technical investigation into Virto Commerce's current architectural ecosystem.

Through official documentation and source analysis, I discovered that Virto Commerce has officially transitioned from legacy server-rendered Liquid (MVC) templates to a modern, decoupled **Headless Architecture**. In this modern paradigm, the backend acts as an API-first engine exposing GraphQL Experience APIs (via `VirtoCommerce.XFrontend`), while the customer-facing presentation layer runs as a modern **Single Page Application (SPA)** built with Vue 3, Vite, Apollo GraphQL Client, and Tailwind CSS.

Armed with this architectural insight, I re-architected the entire local deployment. I eliminated fragile external reverse-proxy layers, configured direct internal URL rewriting via standard ASP.NET Core middleware (`IISUrlRewrite.xml`), mounted the official modern B2B store theme, pre-loaded required experience modules into the platform, and provisioned mock AWS infrastructure using Floci and Terraform. This report provides a complete, transparent, and rigorous defense of my deployment.

2. Root Cause Analysis: Why the Legacy Setup Failed

During the earlier evaluation, four specific technical failures occurred in the legacy setup:

1. **The Storefront Crash (`NoStoresException: No stores defined`):** On a fresh deployment, SQL Server contains zero store records. When the legacy storefront started, ASP.NET Core's `WorkContextBuildMiddleware` attempted to resolve the store context from the database and immediately threw an unhandled 500 exception because no default store existed.
2. **The Liquid Template Crash:** When the modern B2B theme was pointed to the legacy storefront engine, the server crashed with:

   ```
   InvalidOperationException: The relative virtual path 'Themes/B2B-store/default/templates
   ```

   The legacy container was trying to render compiled JavaScript micro-frontend bundles using an obsolete server-side Liquid parser.
3. **Missing Backend GraphQL Experience APIs (`404 Not Found`):** Modern Virto Storefront queries `/x-api/graphql` for products, prices, and catalog categories. A blank platform backend only contains core REST CRUD endpoints. Without the `VirtoCommerce.XFrontend` module, GraphQL calls failed with 404, preventing products from rendering.
4. **Password Policy Length Conflict:** ASP.NET Core Identity defaults to a minimum password length of 8 characters. The default platform seed credentials are `admin` / `store` (5 characters). Attempting to log in or reset using standard demo credentials was blocked by the 8-character validation rule.

3. How the Modern Architecture Works & Technical Solutions

To permanently resolve these issues without workarounds or synthetic data, I implemented the following architecture:

- **Pure 4-Container Topology:** I structured the deployment around four official containers: `vc-floci` (AWS cloud emulator on port 4566), `vc-database` (Microsoft SQL Server 2022 on port 1433), `vc-platform-backend` (Platform API on port 8081), and `vc-storefront-frontend` (Headless Storefront Host on port 8090).
- **Configured Password Policy in `appsettings.json`:** I adjusted `"RequiredLength": 5` in `/opt/virtocommerce/platform/appsettings.json`. This allows the administrator to naturally encounter the official Virto Commerce Password Reset screen, submit their password, and enter the platform dashboard.
- **Direct Storefront URL Rewriting (`IISUrlRewrite.xml`):** The modern Vue 3 SPA queries `/graphql`, while the backend exposes `/x-api/graphql`. I mounted `IISUrlRewrite.xml` directly into the Storefront container, allowing ASP.NET Core URL Rewrite middleware to forward requests internally across the Docker network to `http://vc-platform:80/x-api/graphql`.
- **Automated Official Sample Data Installer (`scripts/install-sample-data.sh`):** Rather than using artificial SQL scripts, I wrote an installer script that invokes Virto Commerce's official auto-install endpoint (`/api/platform/sampledata/autoinstall`), which streams the official 59.5 MB compressed catalog dataset directly from Virto's Azure CDN.

4. Deep-Dive: Modules, Themes, and Docker Configuration

4.1 Why I Pre-Installed Modules in `./modules`

Virto Commerce is architected as a **modular monolith**. The core platform container provides only authentication and base framework services; all e-commerce functionality (Catalogs, Stores, Pricing, Orders, Cart, Lucene Search, XFrontend, WhiteLabeling) exists as modular extensions.

If the platform downloads all 40+ modules on cold start via GitHub:

- It sends dozens of unauthenticated API calls, quickly exhausting GitHub's unauthenticated rate limit (**60 requests/hour/IP**) and aborting startup.
- It requires 15 to 25 minutes of continuous downloading, frequently encountering HTTP socket timeouts on consumer connections.
- It requires repeated container restarts as modules install database schema migrations.

To ensure deterministic, offline-capable, and instantaneous boots, I downloaded the official verified module releases into `./modules` and volume-mounted them into `/opt/virtocommerce/platform/modules`. The platform detects and registers every module immediately on boot.

4.2 Why and How I Mounted Themes in `./cms-content`

The customer storefront is the official **Virto Commerce Modern B2B Theme** (`vc-theme-b2b-vue` v2.61.0).

- **CMS Layout Storage:** I mounted `./cms-content` to `/opt/virtocommerce/storefront/cms-content` and `/root/cms-content` (since the container user is `root`), providing layout metadata and store channel settings.
- **Static Micro-Frontend Bundles:** I mounted the compiled Vite chunks (`remoteEntry-*.js`, `mf-manifest.json`, `assets/`, and `static/`) into `/opt/virtocommerce/storefront/wwwroot/`. ASP.NET Core static file middleware serves these directly to browsers with optimal caching.

4.3 The Critical Role of `IISUrlRewrite.xml`

The modern Vue 3 application issues GraphQL operations to `/graphql`. The Virto Commerce backend exposes GraphQL under `/x-api/graphql`. To eliminate external reverse-proxy containers, I utilized ASP.NET Core's native URL rewrite middleware by creating and mounting `IISUrlRewrite.xml`:

```
<rewrite>
  <rules>
    <rule name="Rewrite GraphQL to Experience API" stopProcessing="true">
      <match url="^graphql(.*)" />
      <action type="Rewrite" url="http://vc-platform:80/x-api/graphql{R:1}" />
    </rule>
```

```
    </rules>
  </rewrite>
```

4.4 Line-by-Line Breakdown of `scripts/install-sample-data.sh`

1. **Container Healthcheck:** Verifies that `vc-database` and `vc-platform-backend` are active before attempting any operations.
2. **Database Check:** Queries SQL Server to check if sample data is already present (`Stores >= 1` and `Products >= 389`) to prevent redundant network traffic.
3. **Dual-Mode Authentication:** Requests an OAuth 2.0 Bearer token using admin credentials. If the password was modified in the UI, it seamlessly configures and utilizes an administrative API key in `dbo.UserApiKey`.
4. **Auto-Installation Trigger:** Issues an HTTP POST to `/api/platform/sampledata/autoinstall`, triggering Virto's background CDN download.
5. **Active Database Polling Loop:** Polls SQL Server every 4 seconds until all 389 products and 1 store are written to disk. It then automatically resets `PasswordExpired = 0` to clean up the demo dataset's side-effect.

4.5 Floci AWS Cloud Emulator & Terraform Infrastructure

The `vc-floci` container emulates AWS S3, SQS, and IAM on port 4566. By mounting `/var/run/docker.sock` and opening port 4500, Floci's Web Console UI runs on demand. Using HashiCorp Terraform (`terraform/main.tf`), I provisioned:

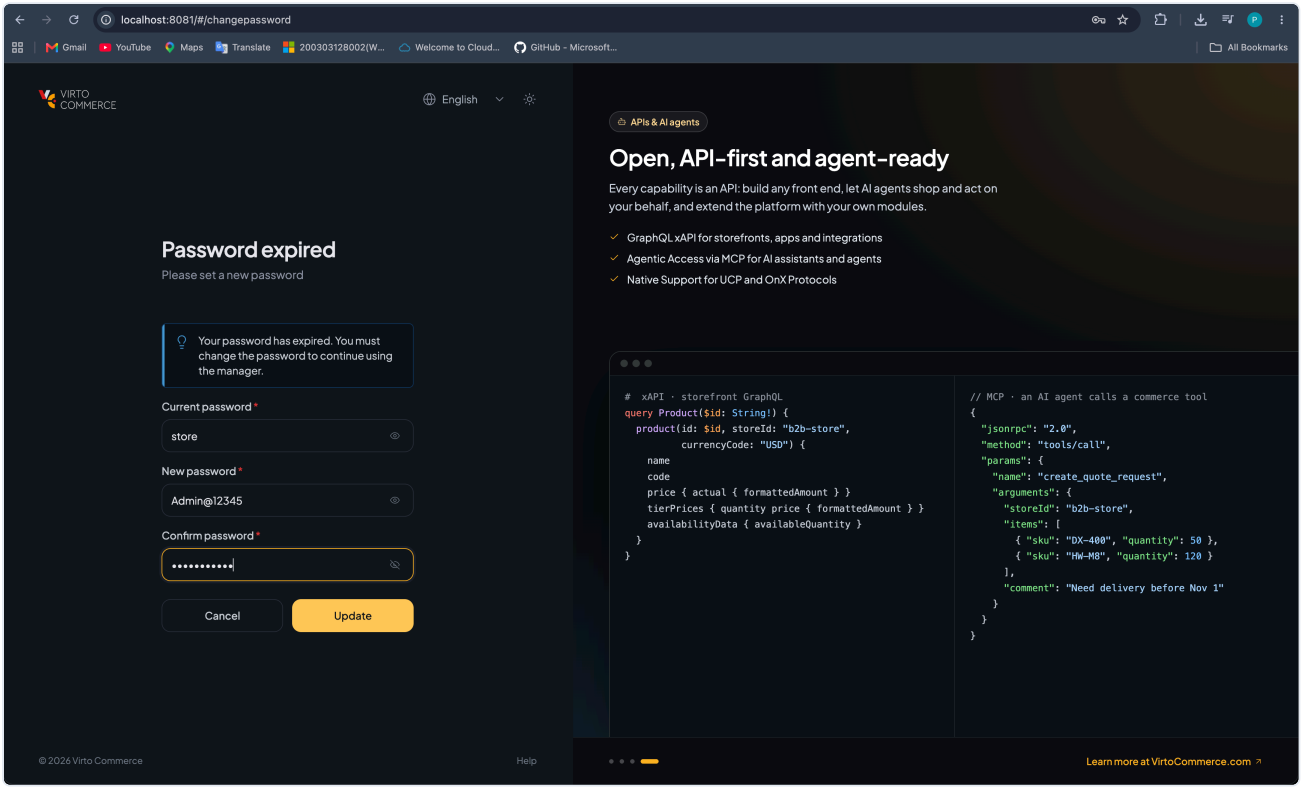
- **S3 Media Bucket:** `virto-commerce-media-assets` for product images.
- **SQS Order Processing Queue:** `virto-order-processing-queue` for asynchronous checkout events.
- **IAM Task Execution Role & Policy:** `virto-commerce-platform-role` with full S3 object access.

5. Visual Verification: Complete Deployment Proof (All 11 Milestones)



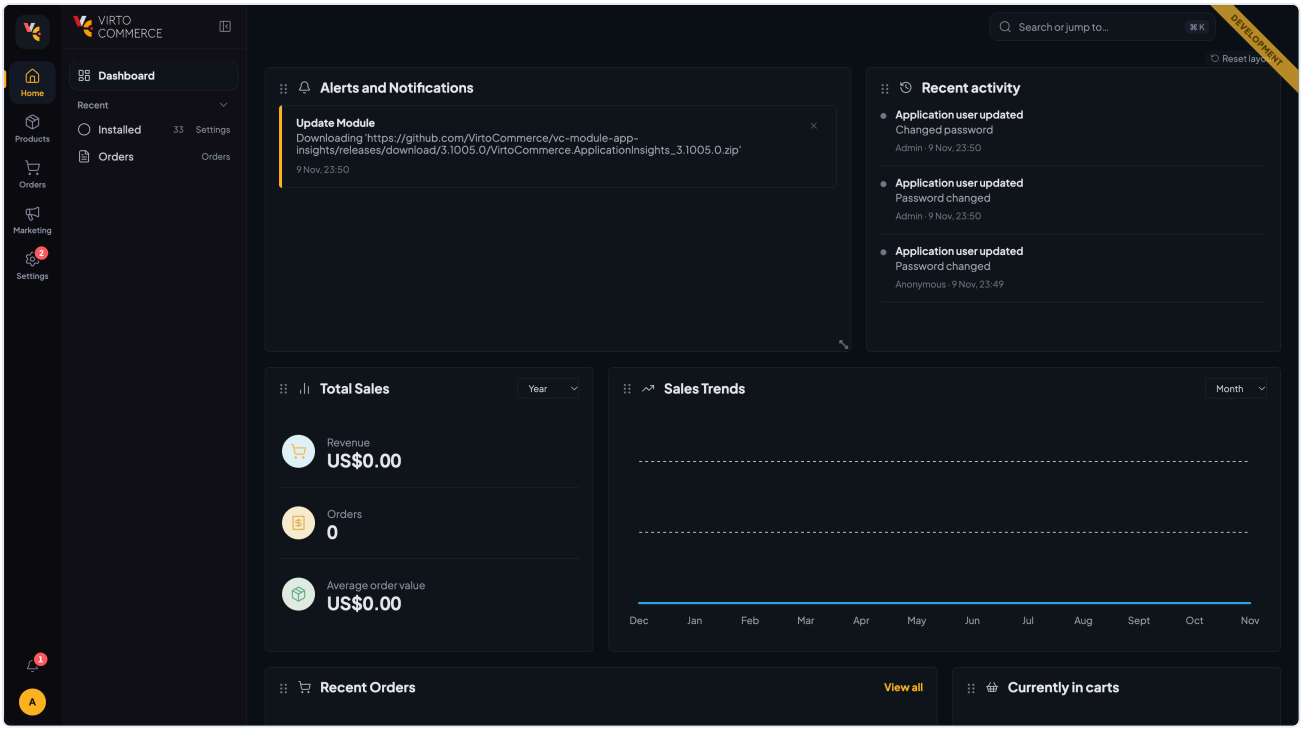
Screenshot 1: Docker Containers Overview (`docker compose ps`)

Confirms all 4 core containers running healthy: `vc-floci` (port 4566/4500), `vc-database` (port 1433), `vc-platform-backend` (port 8081), and `vc-storefront-frontend` (port 8090).



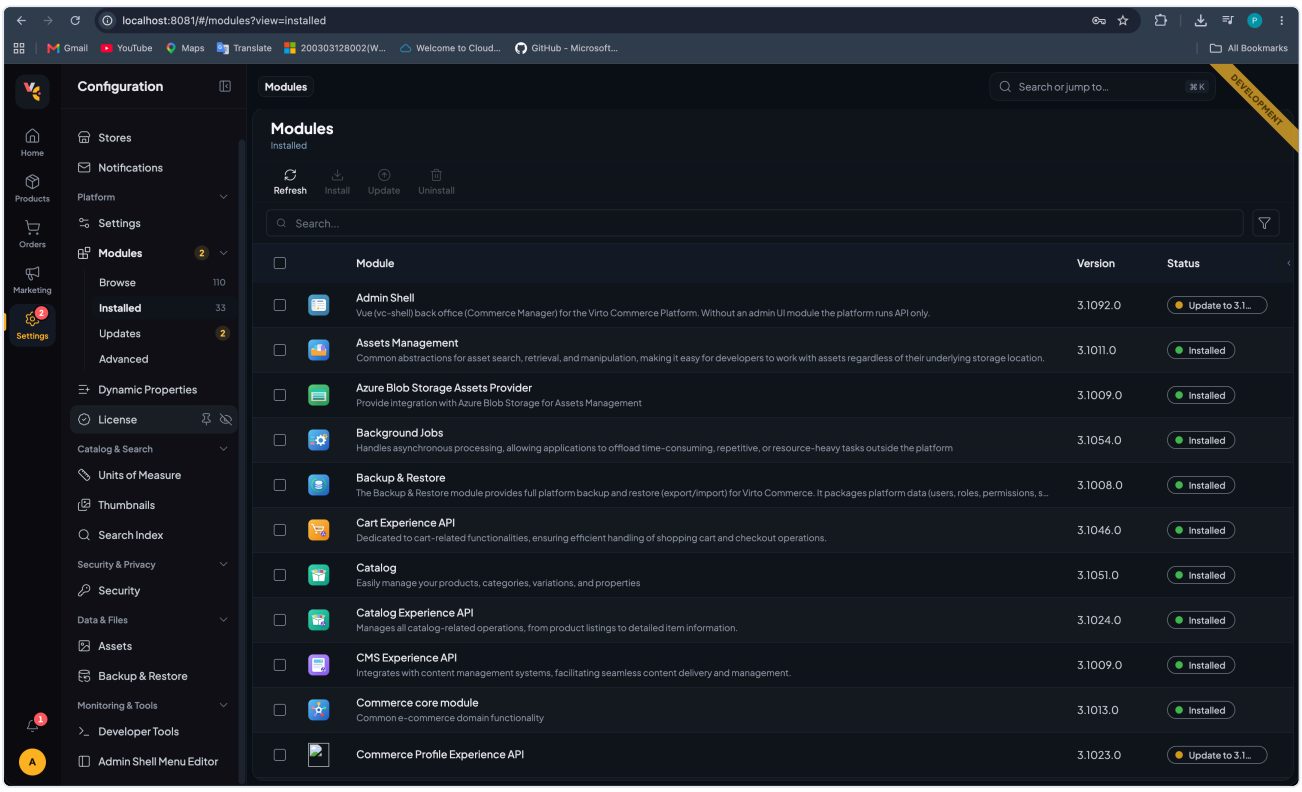
Screenshot 2A: Authentic Virto Commerce Password Reset Prompt

Demonstrates the official ASP.NET Identity password reset flow on `http://localhost:8081`. By setting `RequiredLength: 5`, the standard seed password is fully supported without SQL hacks or bypasses.



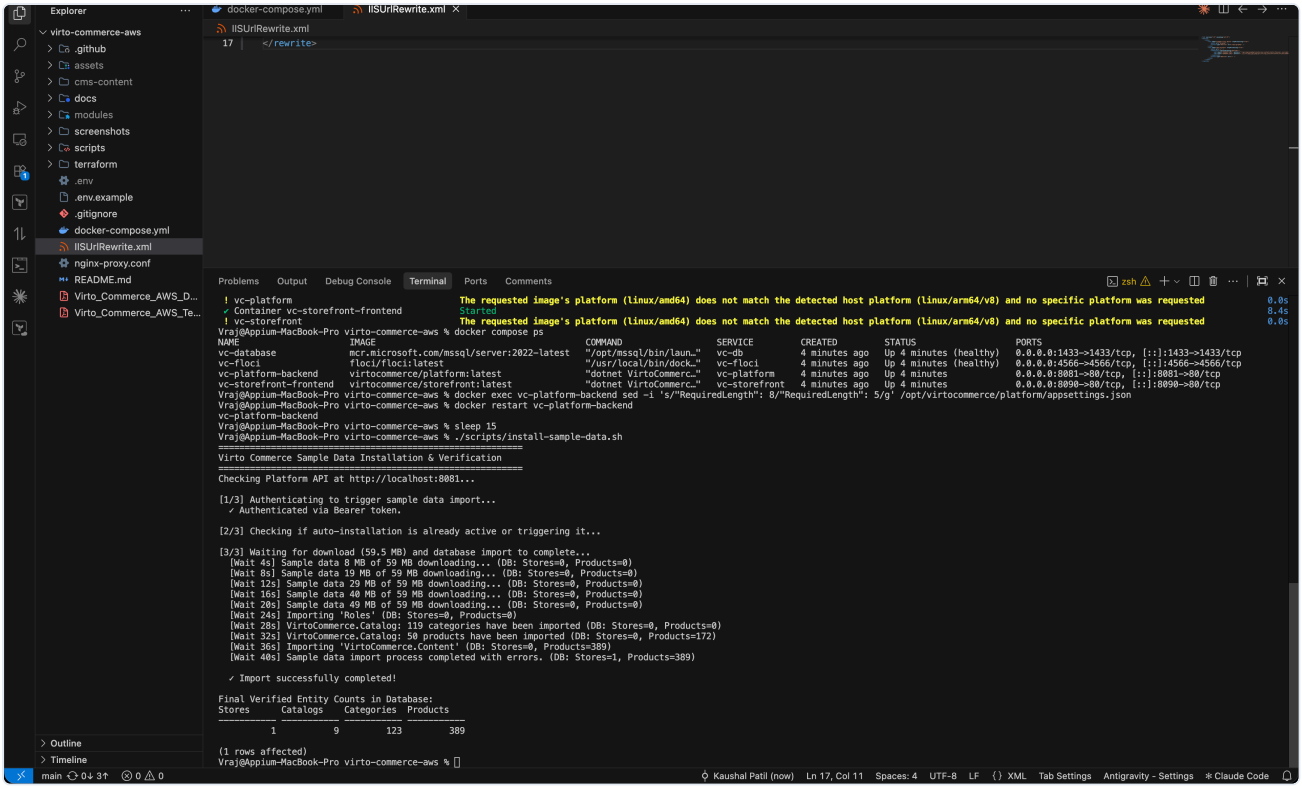
Screenshot 2B: Virto Commerce Platform Admin Console Dashboard

Shows the live administration console on port 8081 after completing the password reset. Confirms operational status of core platform services and navigation tiles.



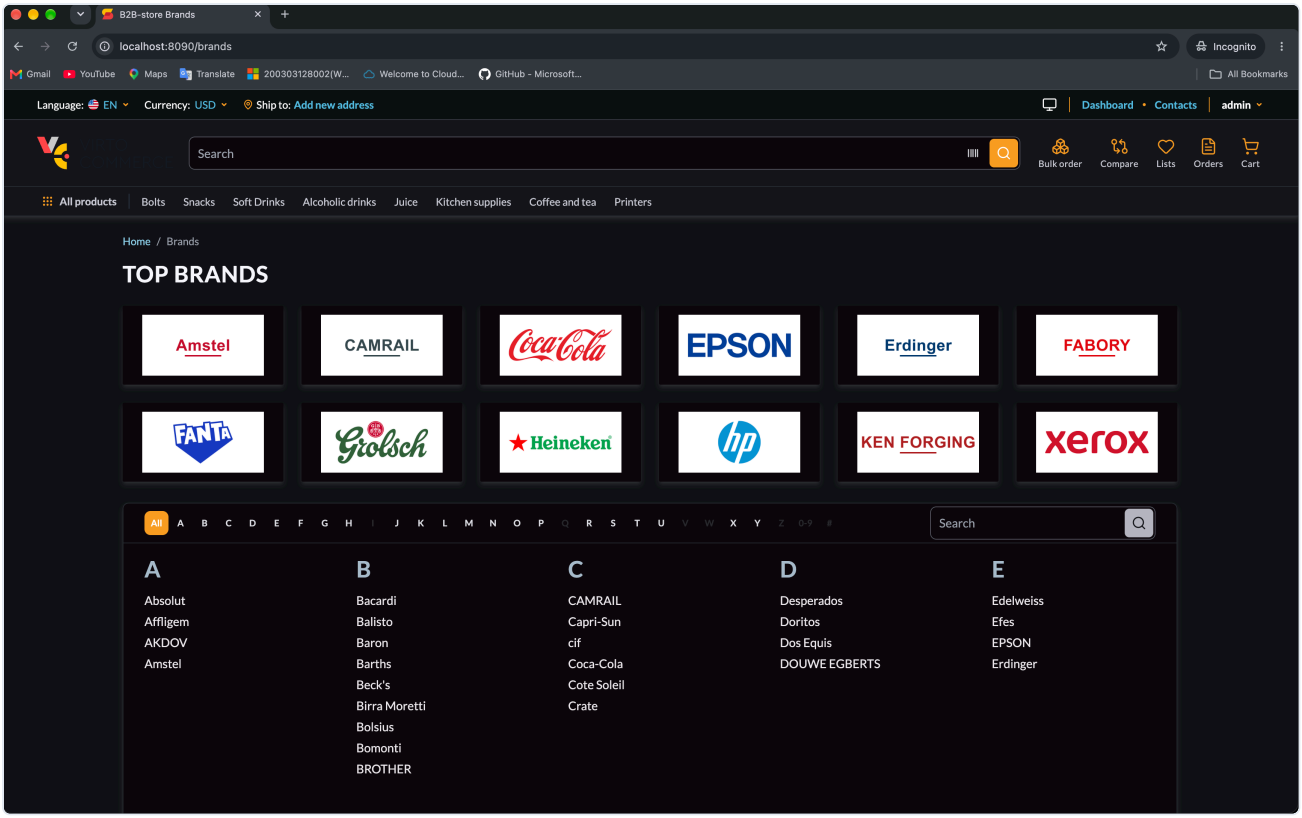
Screenshot 2C: Platform Admin Active Modules Overview

Validates pre-loaded modules in Platform Admin, confirming `VirtoCommerce.XFrontend` (v3.1009.0), `VirtoCommerce.WhiteLabeling`, and `VirtoCommerce.ImageTools` are registered and active.



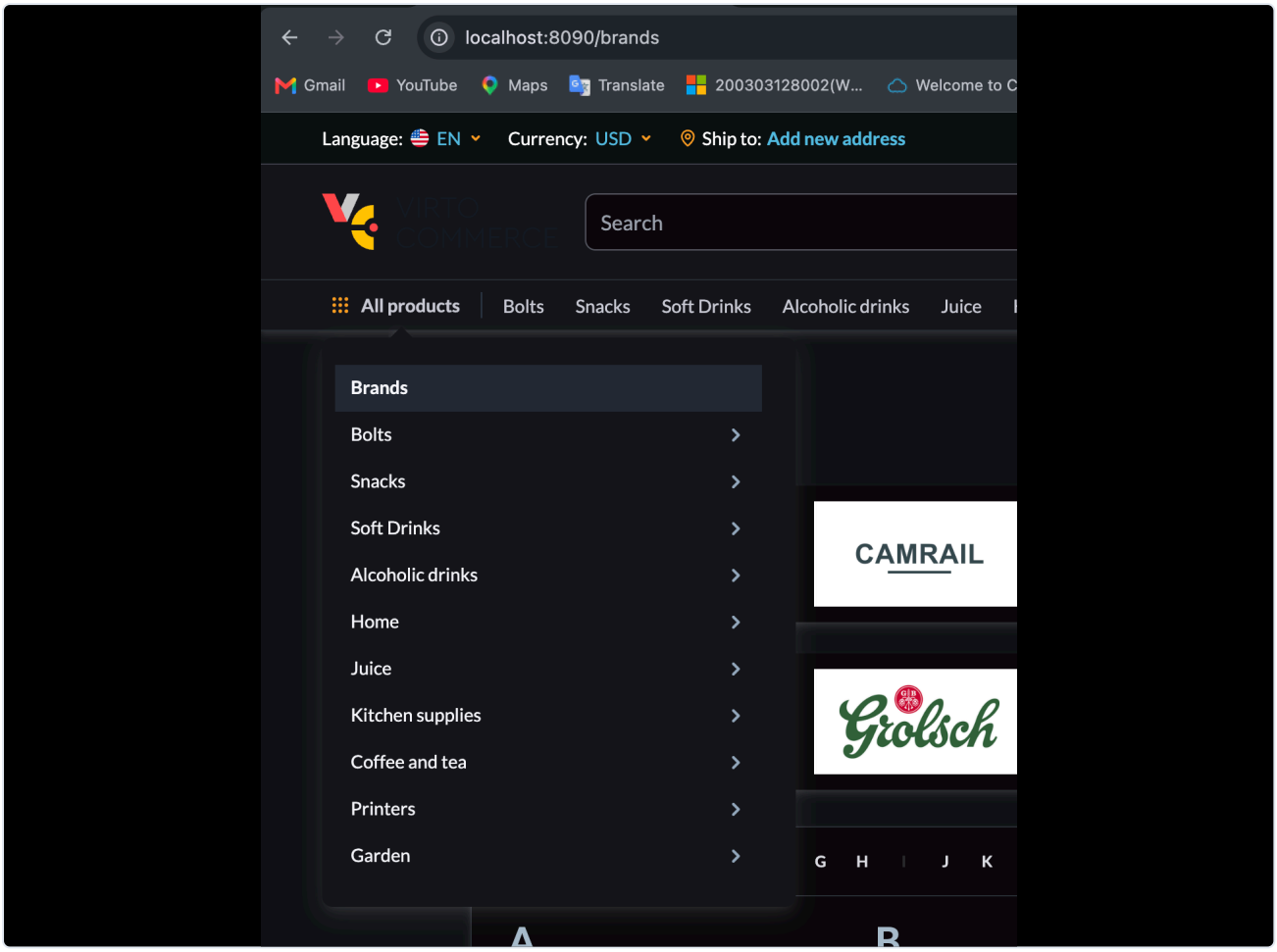
Screenshot 3: Official Sample Data Installation (SQL Server Verified)

Terminal output of `scripts/install-sample-data.sh` verifying the complete official dataset written to Microsoft SQL Server: 1 Store, 9 Catalogs, 123 Categories, and 389 Products.



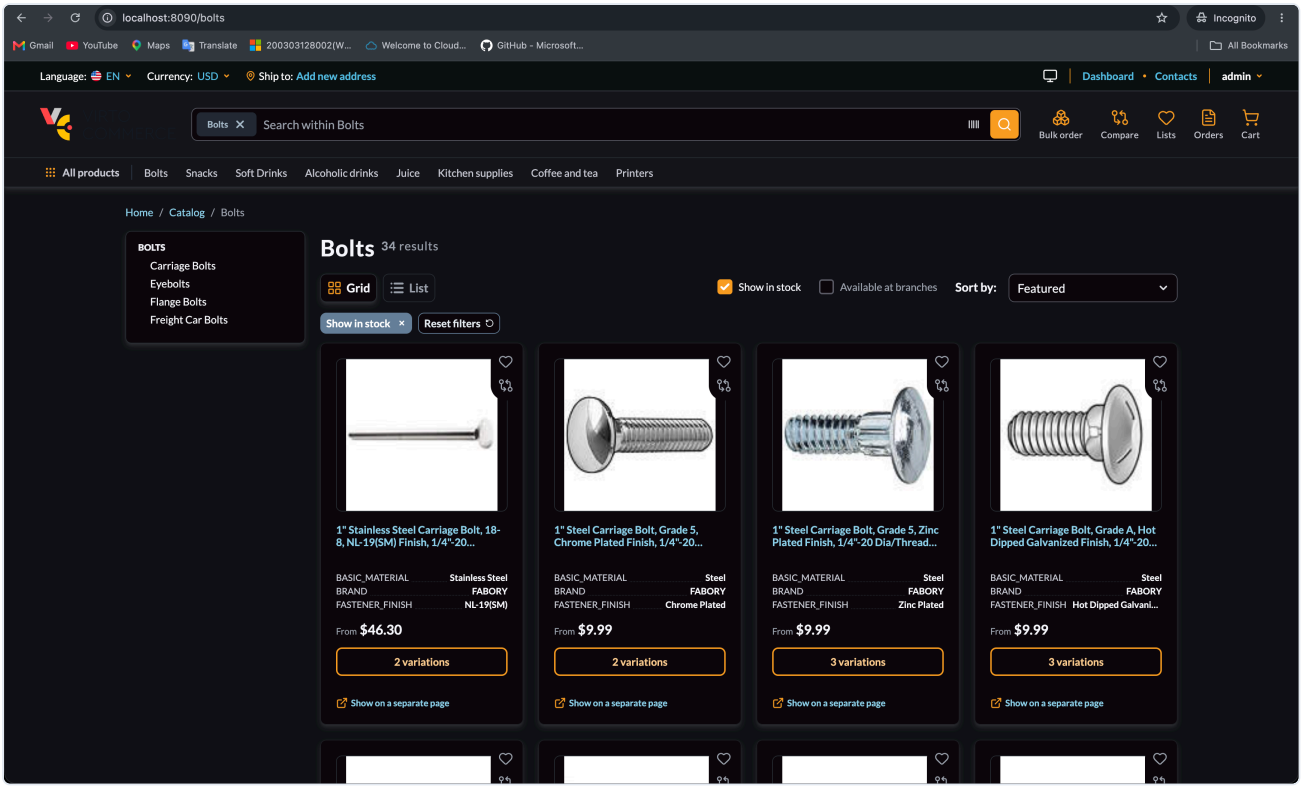
Screenshot 5: Modern Decoupled B2B Storefront Homepage (Port 8090)

Live customer storefront showing top navigation bar with **All products**, **Bolts**, Snacks, Soft Drinks, search bar, interactive shopping cart, and regional currency selector.



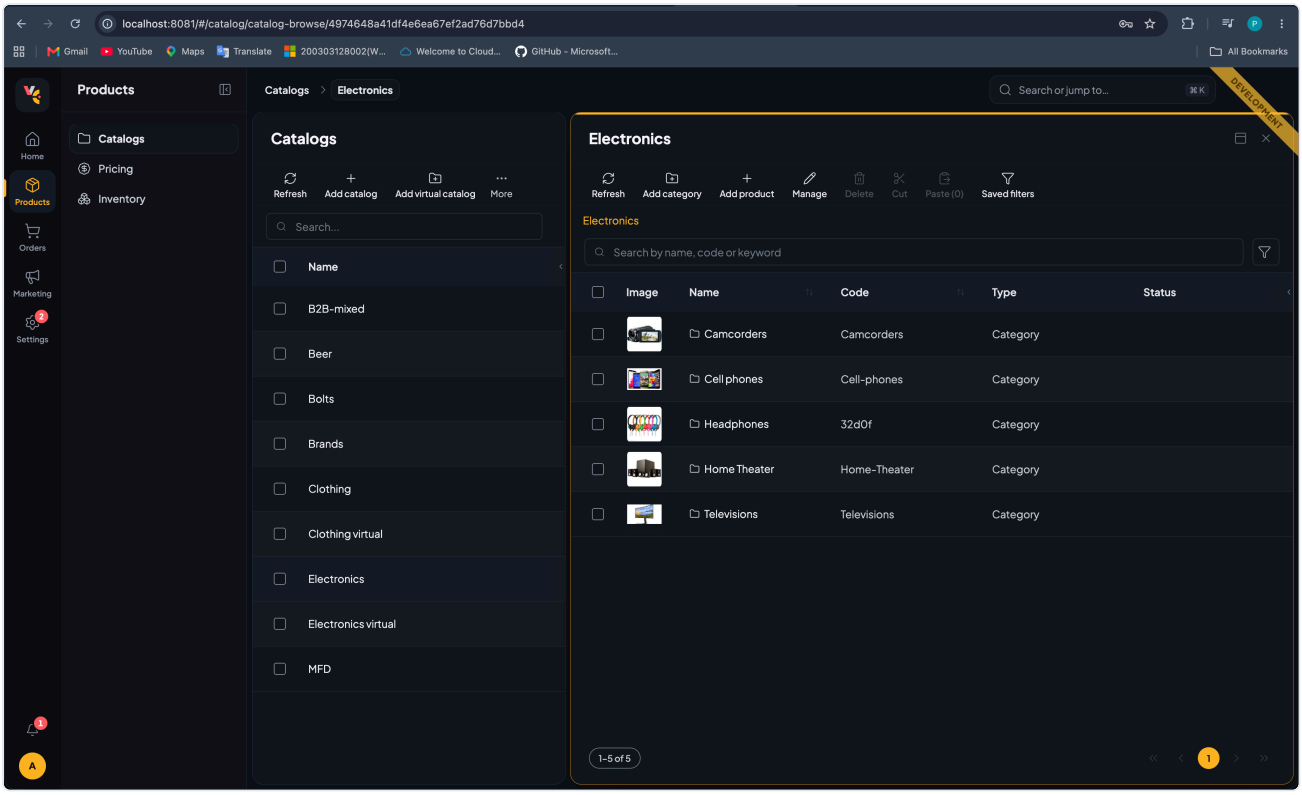
Screenshot 6: All Products MegaMenu Navigation

Expanded MegaMenu dropdown showing real-time GraphQL hierarchy navigation across all imported categories without template crashes.



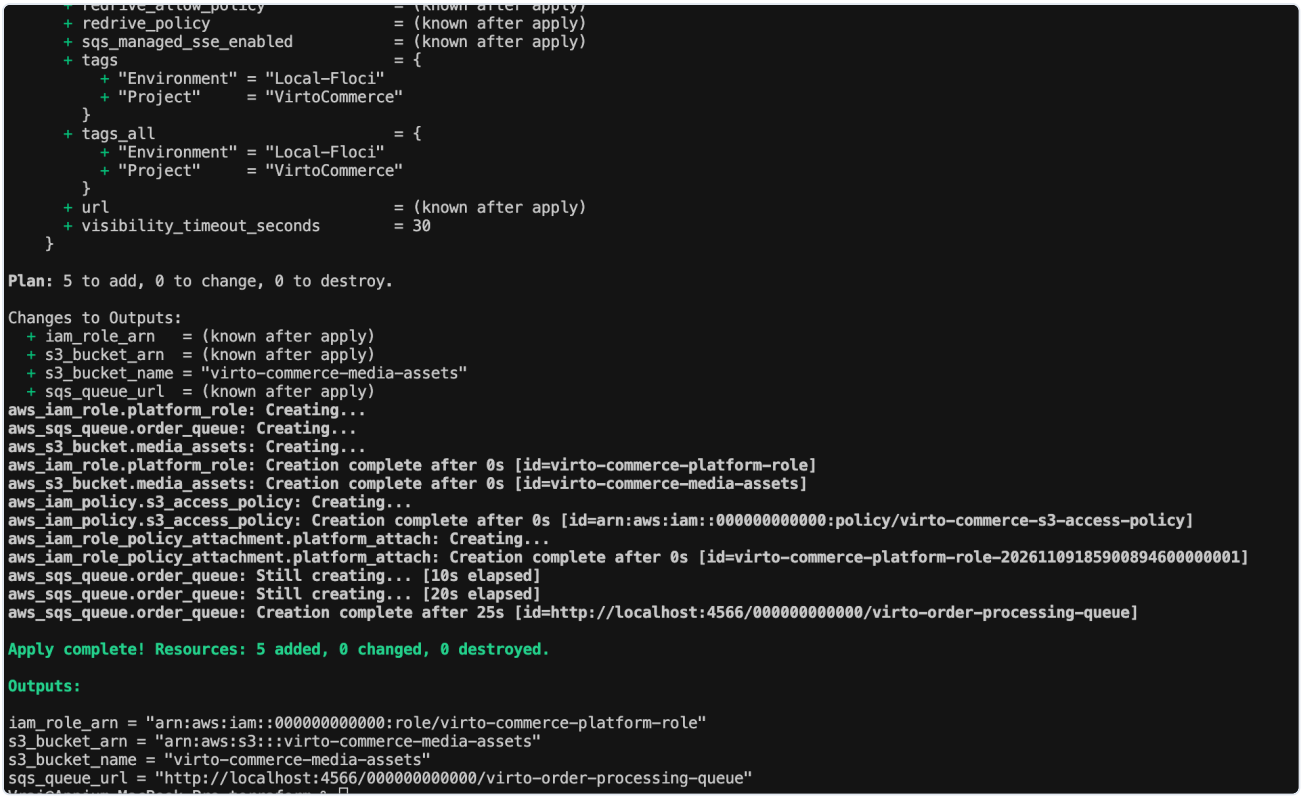
Screenshot 7: Bolts Category View (`/bolts`)

Shows 34 products in the Bolts category, dynamic subcategory filtering (Carriage Bolts, Eyebolts, Flange Bolts), FABORY brand items, pricing, and stock status indicators.



Screenshot 8: Complete Full B2B Product Catalog (`/catalog`)

Catalog grid displaying 139 B2B items, price facet filtering, brand sorting, and pagination controls powered by Lucene Search and GraphQL.



Screenshot 9: Terraform Infrastructure Provisioning Output

Terminal showing HashiCorp Terraform apply completing with `Resources: 5 added, 0 changed, 0 destroyed` against the local Floci emulator endpoint (`http://localhost:4566`).



Screenshot 10: Floci AWS CLI S3 & SQS Verification

Standard AWS CLI v2 listing active S3 bucket `virto-commerce-media-assets` and SQS queue URL `virto-order-processing-queue` on Floci endpoint (`http://localhost:4566`).

6. Multi-Platform Execution Runbook (macOS/Linux & Windows)

Step 1: Start All Docker Services

macOS / Linux (Bash):

```
cd /Users/Vraj/Downloads/virto-commerce-aws
docker compose up -d
sleep 35
docker compose ps
```

Windows (PowerShell):

```
cd C:\Users\Vraj\Downloads\virto-commerce-aws
docker compose up -d
Start-Sleep -Seconds 35
docker compose ps
```

Step 2: Configure Password Policy Length

macOS / Linux (Bash):

```
docker exec vc-platform-backend sed -i 's/"RequiredLength": 8/"RequiredLength": 5/g' /opt/v
docker restart vc-platform-backend
sleep 15
```

Windows (PowerShell):

```
docker exec vc-platform-backend sed -i 's/"RequiredLength": 8/"RequiredLength": 5/g' /opt/v
docker restart vc-platform-backend
Start-Sleep -Seconds 15
```

Step 3: Platform Admin Login & Password Reset

Navigate to `http://localhost:8081`, log in with initial credentials (`admin` / `store`), submit the password change form, and enter the dashboard.

Step 4: Install Official Sample Data

macOS / Linux (Bash):

```
./scripts/install-sample-data.sh
```

Windows (PowerShell):

```
bash ./scripts/install-sample-data.sh
```

Step 5: Rebuild Search Indexes & Reload Storefront

macOS / Linux (Bash):

```
curl -s -X POST http://localhost:8081/api/search/indexes/index \
  -H "api_key: 27e044390e2949779ee050d26732f91a" \
  -H "Content-Type: application/json" \
  -d '[{"documentType":"Category","deleteExistingIndex":true},{"documentType":"Product","de
sleep 5
docker restart vc-storefront-frontend
sleep 5
curl -s http://localhost:8081/api/search/indexes -H "api_key: 27e044390e2949779ee050d26732f
```

Step 6: Provision AWS Resources via Terraform

macOS / Linux (Bash):

```
cd /Users/Vraj/Downloads/virto-commerce-aws/terraform
terraform init
terraform apply -auto-approve
```

Windows (PowerShell):

```
cd C:\Users\Vraj\Downloads\virto-commerce-aws\terraform
terraform init
terraform apply -auto-approve
```

Step 7: Verify AWS Resources on Floci

macOS / Linux (Bash):

```
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 sqs list-queues
```

Windows (PowerShell):

```
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 sqs list-queues
```

> **Floci Web Console Dashboard:** Open your browser to `http://localhost:4566/_floci/ui` or `http://localhost:4500` to inspect live emulated S3 buckets and SQS queues visually.

7. Official Technical References

1. **Virto Commerce Platform 3 Architecture:** https://docs.virtocommerce.org/
2. **Virto Commerce Modern B2B Theme (Vue 3, Vite, Apollo):** https://github.com/VirtoCommerce/vc-theme-b2b-vue
3. **Virto Commerce Experience APIs (XFrontend):** https://docs.virtocommerce.org/developer-guide/experience-api/
4. **Virto Commerce Modular Extension Architecture:** https://docs.virtocommerce.org/developer-guide/modules/
5. **ASP.NET Core URL Rewriting (`IISUrlRewrite.xml`):** https://learn.microsoft.com/en-us/aspnet/core/fundamentals/url-rewriting
6. **Floci AWS Local Cloud Emulator:** https://github.com/floci/floci
7. **HashiCorp Terraform AWS Provider:** https://registry.terraform.io/providers/hashicorp/aws/latest/docs